

[AppName] — Startup Plan

Cross-platform professional communication platform: WhatsApp UX · Slack structure · Classroom-style boards · Discord voice

Version: 0.1 — Internal

Team: 3 people

Status: Pre-MVP, Pre-Funding

1. Problem

WhatsApp dominates professional communication in India — schools, coaching institutes, SMBs, freelancer teams, local businesses. No one migrates off it because every alternative (Slack, Teams) has high friction, requires corporate email accounts, and is desktop-first.

The result: classroom announcements in group chats, business files buried in scroll, zero accountability, zero structure.

2. Vision

A mobile-first, phone-number-based professional workspace that feels like WhatsApp, structures like Slack, distributes content like Google Classroom, and supports voice like Discord — permanently free for small teams.

3. Target Market (MVP)

Primary: Indian coaching institutes and schools (K-12, competitive exam prep)

Secondary: Indian SMBs and freelancer teams under 50 people

Why: Highest WhatsApp abuse, lowest Slack adoption, most price-sensitive — perfect for a free-tier wedge

4. Core Feature Set

MVP (Ship This, Nothing Else)

Feature	Inspiration	Description
Phone OTP Registration	WhatsApp	No email, no corp account, just a number
1:1 Messaging	WhatsApp	Real-time, media support

Feature	Inspiration	Description
Workspaces	Slack	Org-level container, invite-only or link-based
Channels	Slack	Topic-based text rooms within a workspace
Boards	Google Classroom	Structured announcement posts — admin posts, members acknowledge/reply, file attachments
File & Image Sharing	WhatsApp	Upload via R2 presigned URLs, in-chat previews
Basic Roles	Discord (lite)	Admin / Member — nothing granular yet
Push Notifications	FCM	Message and board alerts
Cross-platform	—	Web (Flutter Web) + iOS + Android from single codebase

V2 (Post First 100 Users)

- Voice rooms (WebRTC via Cloudflare Calls)
- Granular Discord-style roles and permissions per channel
- Message search (full-text via PostgreSQL (tsvector))
- Board submission tracking (who acknowledged, who hasn't)
- Workspace analytics for admins

V3 (Post Funding)

- End-to-end encryption
- WhatsApp contact/group migration tool
- AI layer — thread summarization, action item extraction
- Custom domain workspaces
- API for third-party integrations

5. Tech Stack

Backend

Layer	Technology	Reason
Language	Rust	Perf moat, correctness guarantees, long-term defensibility

Layer	Technology	Reason
HTTP + WebSocket	Axum 0.7	Best Rust web framework, native WebSocket support
Async Runtime	Tokio (full features)	Industry standard, battle-tested
Database ORM	SQLx (async, PostgreSQL)	Compile-time query checking, no ORM overhead
Auth	JWT (jsonwebtoken crate)	Stateless session tokens
Media	AWS S3 SDK (R2-compatible)	Presigned URL uploads, zero proxying
Serialization	Serde + serde_json	Standard

```
toml

# Cargo.toml - core deps
[
dependencies
]
axum = "0.7"
tokio = { version = "1", features = ["full"] }
sqlx = { version = "0.7", features = ["runtime-tokio", "postgres", "uuid", "chrono"] }
serde = { version = "1", features = ["derive"] }
serde_json = "1"
jsonwebtoken = "9"
aws-sdk-s3 = "1"
aws-config = "1"
redis = { version = "0.24", features = ["tokio-comp"] }
uuid = { version = "1", features = ["v4"] }
chrono = { version = "0.4", features = ["serde"] }
tower = "0.4"
tower-http = { version = "0.5", features = ["cors", "trace"] }
tracing = "0.1"
tracing-subscriber = "0.3"
```

Frontend

Layer	Technology
Framework	Flutter (Dart)
Targets	Web · iOS · Android (single codebase)
State Management	Riverpod

Layer	Technology
WebSocket Client	<code>web_socket_channel</code>
HTTP Client	<code>dio</code>
Push Notifications	<code>firebase_messaging</code>
Local Storage	<code>flutter_secure_storage</code>
Media Handling	<code>image_picker</code> , <code>file_picker</code>
Navigation	<code>go_router</code>

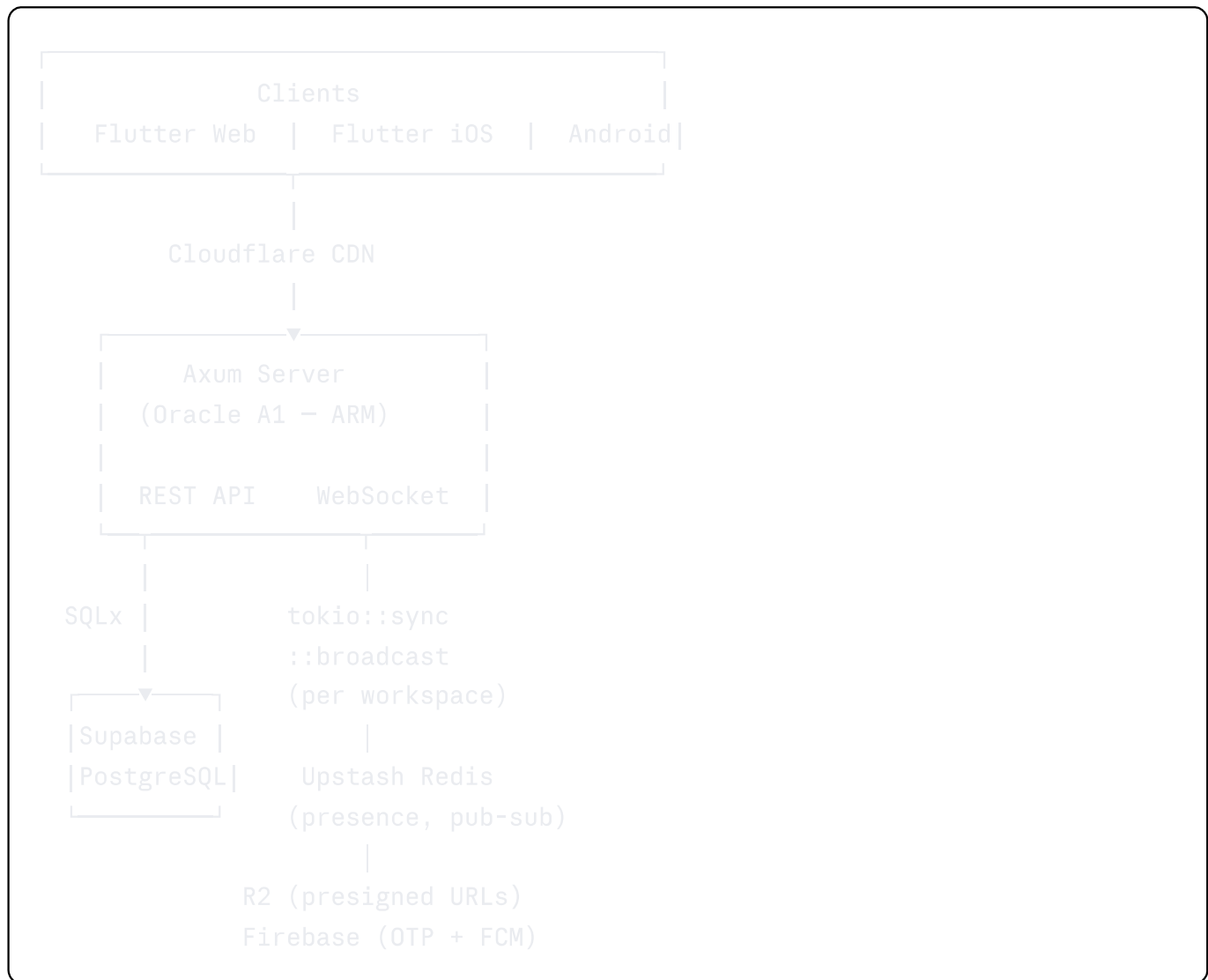
Infrastructure

Layer	Service	Free Tier
Compute	Oracle Cloud Always Free (Ampere A1)	4 ARM cores, 24GB RAM
Database	Supabase	500MB PostgreSQL + built-in Realtime
Cache / Pub-Sub	Upstash Redis	10K req/day (enough for MVP)
Media Storage	Cloudflare R2	10GB + zero egress fees
CDN	Cloudflare Free	Unlimited bandwidth
Phone Auth + SMS OTP	Firebase Auth	10K SMS/month free
Push Notifications	Firebase FCM	Free, no cap
Web Hosting	Cloudflare Pages	Free, unlimited requests
Voice (V2)	Cloudflare Calls	Free beta tier
CI/CD	GitHub Actions	2,000 min/month free

Total monthly cost at MVP: ₹0

6. Architecture

High-Level



WebSocket Message Flow



Media Upload Pattern (Never proxy through compute)



Database Schema (Core Tables)



```
-- Users
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone       TEXT UNIQUE NOT NULL,
  display_name TEXT NOT NULL,
  avatar_url  TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Workspaces
CREATE TABLE workspaces (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  slug        TEXT UNIQUE NOT NULL,
  owner_id    UUID REFERENCES users(id),
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Workspace members
CREATE TABLE workspace_members (
  workspace_id UUID REFERENCES workspaces(id),
  user_id      UUID REFERENCES users(id),
  role         TEXT DEFAULT 'member', -- 'admin' | 'member'
  joined_at    TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (workspace_id, user_id)
);

-- Channels
CREATE TABLE channels (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  workspace_id UUID REFERENCES workspaces(id),
  name        TEXT NOT NULL,
  type        TEXT DEFAULT 'text', -- 'text' | 'board'
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Messages
CREATE TABLE messages (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  channel_id  UUID REFERENCES channels(id),
  sender_id   UUID REFERENCES users(id),
  content     TEXT,
  media_url   TEXT, -- R2 object key
  thread_of   UUID REFERENCES messages(id), -- for Slack-style threads
  created_at  TIMESTAMPTZ DEFAULT now()
);
```

```

-- Boards (Classroom-style posts)
CREATE TABLE board_posts (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  channel_id  UUID REFERENCES channels(id),
  author_id   UUID REFERENCES users(id),
  title       TEXT NOT NULL,
  body        TEXT,
  attachment_url TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- Board acknowledgements
CREATE TABLE board_acknowledgements (
  post_id  UUID REFERENCES board_posts(id),
  user_id  UUID REFERENCES users(id),
  acked_at TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (post_id, user_id)
);

-- Direct messages
CREATE TABLE direct_messages (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  sender_id   UUID REFERENCES users(id),
  receiver_id UUID REFERENCES users(id),
  content     TEXT,
  media_url   TEXT,
  created_at  TIMESTAMPTZ DEFAULT now()
);

```

7. Team & Roles

Person	Role	Owns
You	Founder + Backend Lead	Rust/Axum server, WebSocket infra, database schema, architecture decisions, DevOps
Flutter Dev	Frontend Lead	Full Flutter codebase — Web, iOS, Android; UI components, state management, API integration
Dev 3	Ops / Tooling	GitHub Actions CI/CD pipelines, Oracle server setup, Firebase config, Supabase setup, environment management

Dev 3 Specific Tasks (CI/CD)

- Set up GitHub Actions: lint → test → build → deploy pipeline
 - Flutter Web build → auto-deploy to Cloudflare Pages on `main` push
 - Rust build checks + `cargo test` on PRs
 - Manage `.env` secrets in GitHub Secrets
 - Oracle server provisioning (Ubuntu 22.04, Nginx reverse proxy, SSL via Certbot)
 - Supabase migrations via `sqlx migrate run` in pipeline
-

8. Development Roadmap

Phase 0 — Foundation (Weeks 1-2)

Owner: All 3

- ☐ Oracle A1 instance provisioned (Ubuntu 22.04)
- ☐ Nginx + SSL (Certbot / Cloudflare Origin Cert) configured
- ☐ Supabase project created, schema migrations set up
- ☐ Upstash Redis instance created
- ☐ Cloudflare R2 bucket created, CORS configured
- ☐ Firebase project created (Auth + FCM)
- ☐ GitHub monorepo structure: `/backend` (Rust) + `/app` (Flutter)
- ☐ GitHub Actions: Rust CI + Flutter Web deploy to CF Pages
- ☐ Base Axum project: health check, JWT middleware, CORS

Phase 1 — Auth + Messaging Core (Weeks 3-5)

Backend (You):

- ☐ Phone OTP via Firebase Auth REST API
- ☐ JWT issue / refresh / revoke
- ☐ User profile CRUD
- ☐ 1:1 messaging REST endpoints
- ☐ WebSocket server with JWT handshake auth
- ☐ `tokio::broadcast` channel manager (per workspace)
- ☐ Redis presence tracking (online/offline/last seen)
- ☐ R2 presigned URL endpoint

Frontend (Flutter Dev):

- ☐ Onboarding flow — phone input → OTP → display name
- ☐ Home screen skeleton
- ☐ 1:1 chat screen — real-time WebSocket messages
- ☐ Image/file send via presigned URL

- ☐ Push notification integration (FCM)

Phase 2 — Workspaces + Channels (Weeks 6-8)

Backend:

- ☐ Workspace create/join/invite endpoints
- ☐ Channel CRUD within workspace
- ☐ Channel WebSocket subscriptions
- ☐ Role enforcement middleware (admin vs member)

Frontend:

- ☐ Workspace creation + invite link flow
- ☐ Channel sidebar (Slack-style)
- ☐ Channel chat view
- ☐ Admin panel (basic: manage members, create channels)

Phase 3 — Boards (Weeks 9-10)

Backend:

- ☐ Board post CRUD endpoints
- ☐ Board acknowledgement endpoint
- ☐ Attachment handling via R2
- ☐ Push notification on new board post

Frontend:

- ☐ Board channel type (distinct UI from text channel)
- ☐ Post creation form (title, body, attachment)
- ☐ Post feed view with acknowledge button
- ☐ Acknowledgement count visible to admins

Phase 4 — Polish + First Users (Weeks 11-12)

- ☐ Bug fixes, performance pass
- ☐ Proper error states and loading skeletons
- ☐ Onboarding tutorial (first-time workspace creation)
- ☐ App icons, splash screens, web favicon
- ☐ Privacy policy, terms of service (required for app stores)
- ☐ TestFlight (iOS) + Google Play Internal Track (Android)
- ☐ Cloudflare Pages (Web)
- ☐ **Get 1 real school or coaching institute using it**

9. MVP Definition

MVP is done when:

1. A teacher can create a workspace, invite students via link
 2. Students join with phone number only
 3. Teacher posts a Board announcement with a PDF attachment
 4. Students receive push notification, open it, acknowledge
 5. Teacher can see who acknowledged and who hasn't
 6. Students can message each other in a channel
 7. All of this works on Web, iOS, and Android
-

10. Go-To-Market

Month 1-2: One coaching institute in Lucknow — personal network, offer white-glove onboarding, free forever for them. Get 50-200 students using it daily.

Month 3-4: Use that cohort as case study. Approach 5 more institutes. Word of mouth in the ed-tech space travels fast locally.

Month 5-6: Product-market fit signals → begin drafting seed pitch.

What NOT to do:

- Don't cold pitch to schools without a working product
 - Don't spend time on landing pages and branding before users
 - Don't launch on Product Hunt before you have retention data
-

11. Funding Path

Pre-seed target: ₹50L-₹1.5Cr (~\$60K-\$180K)

Target investors:

- Antler India (pre-product, takes equity early)
- Venture Catalysts
- 100X.VC (India-focused micro VC, cheque sizes match)
- Sequoia Spark (if traction is strong)

Pitch narrative: "WhatsApp is running India's schools and businesses. We're fixing that — same UX, professional structure, free for small teams. 200 students at [Institute] switched in 2 weeks."

Traction > deck. Don't fundraise until you have a retention story.

12. Risk Register

Risk	Severity	Mitigation
Rust backend velocity too slow	High	Timebox features hard. If stuck >3 days, use pseudocode-level GPT assist, ship it, refactor later
Flutter Web performance issues	Medium	WebSocket on web is tricky; test early on Chrome + Safari
Firebase SMS quota hit	Low	10K/month is plenty for MVP; upgrade if needed
Oracle instance instability	Medium	Daily PostgreSQL dump to R2 from day 1
WhatsApp inertia	High	Don't fight it — target new groups that don't have an existing WhatsApp workflow yet (new semester, new batch)
Feature creep	Critical	Enforce the MVP feature list. No voice rooms, no E2E, no AI until Phase 4 ships

13. Repo Structure

```
/
├── backend/                                # Rust — Axum server
│   ├── src/
│   │   ├── main.rs
│   │   ├── routes/                        # REST handlers
│   │   ├── ws/                           # WebSocket handler + channel manager
│   │   ├── db/                           # SQLx queries
│   │   ├── auth/                         # JWT + Firebase OTP verification
│   │   ├── storage/                      # R2 presigned URL logic
│   │   └── models/                       # Serde structs
│   ├── migrations/                       # SQLx migration files
│   └── Cargo.toml
├── app/                                   # Flutter — cross-platform
│   ├── lib/
│   │   ├── main.dart
│   │   ├── features/
│   │   │   ├── auth/
│   │   │   └── messaging/
```

```
| | | └─ workspace/
| | | └─ channels/
| | | └─ boards/
| | └─ shared/      # Shared widgets, theme, constants
| | └─ services/    # API client, WebSocket client, FCM
| └─ pubspec.yaml
|
└─ .github/
    └─ workflows/
        └─ backend-ci.yml    # Rust lint + test
        └─ flutter-deploy.yml # Flutter web → Cloudflare Pages
└─ README.md
```

14. Key Resources

- [Axum WebSocket Example](#) — MVP WebSocket backbone
 - [Cloudflare R2 Presigned URLs](#) — media upload pattern
 - [Oracle Always Free](#) — Ampere A1 — 24GB RAM, free forever
 - [SQLx Migrations](#) — DB migration setup
 - [Flutter Riverpod Docs](#) — state management
 - [Firebase Phone Auth REST API](#) — OTP without Firebase SDK on backend
 - [Upstash Redis](#) — serverless Redis free tier
 - [100X.VC](#) — India pre-seed fund worth applying to early
-

Last updated: June 2026 | Status: Planning